

Deep Learning

Deep Learning for Time Series



Prof. Anxiao (Andrew) Jiang

Learning Objectives:

1. Understand RNN and LSTM
2. Understand how to use RNN for practical applications



Roadmap of this lecture:

1. Recurrent neural network (RNN)

2. Use RNN for temperature forecasting

2.1 Try 1: Use an RNN

2.2 Try 2: Use LSTM with recurrent dropout

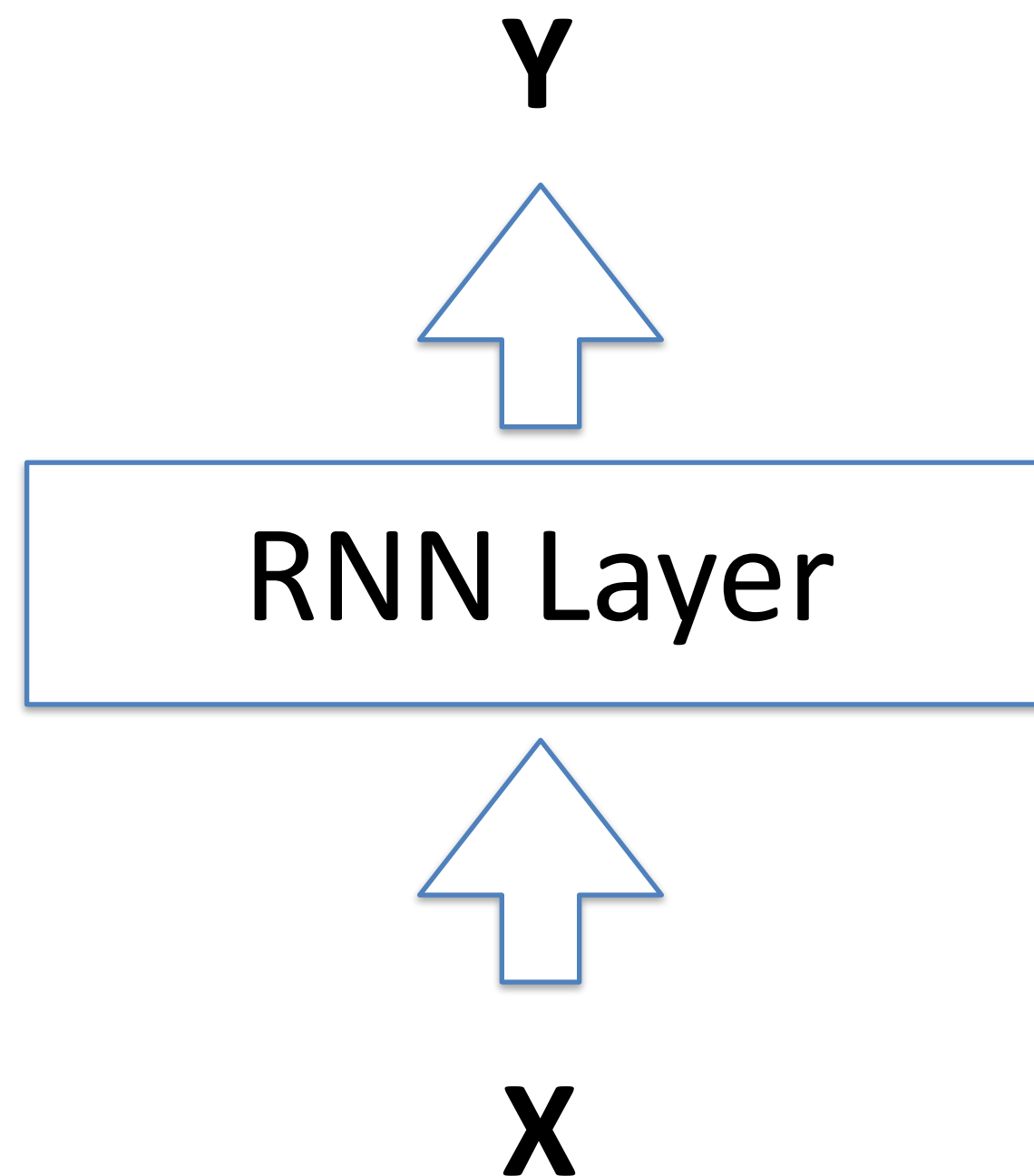
2.3 Try 3: Stack RNN layers

2.4 Try 4: Use bidirectional RNN

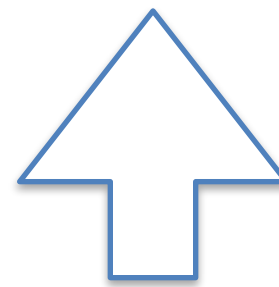


Recurrent Neural Network

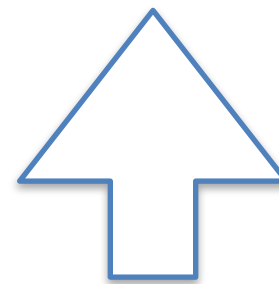
Recurrent Neural Network (RNN)



$$\mathbf{Y} = (y^{(1)} \ y^{(2)} \ y^{(3)} \ \dots \ y^{(T)})$$

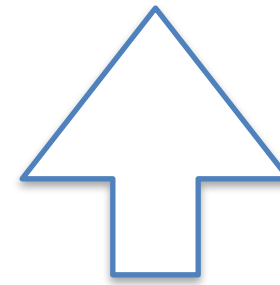


RNN Layer

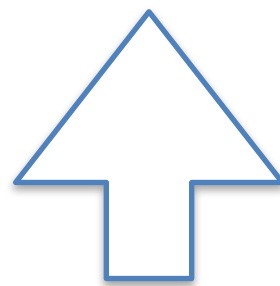
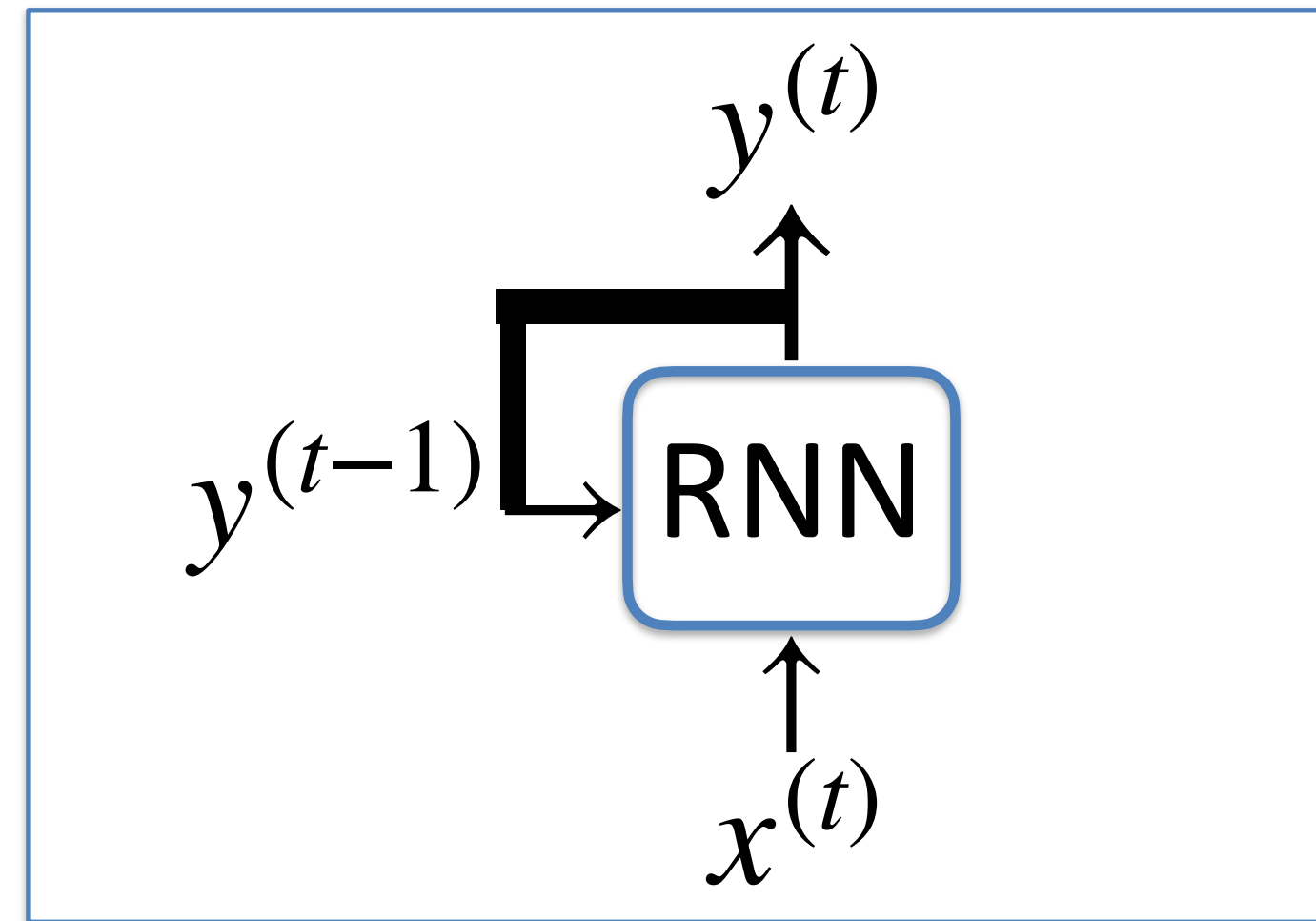


$$\mathbf{X} = (x^{(1)} \ x^{(2)} \ x^{(3)} \ \dots \ x^{(T)})$$

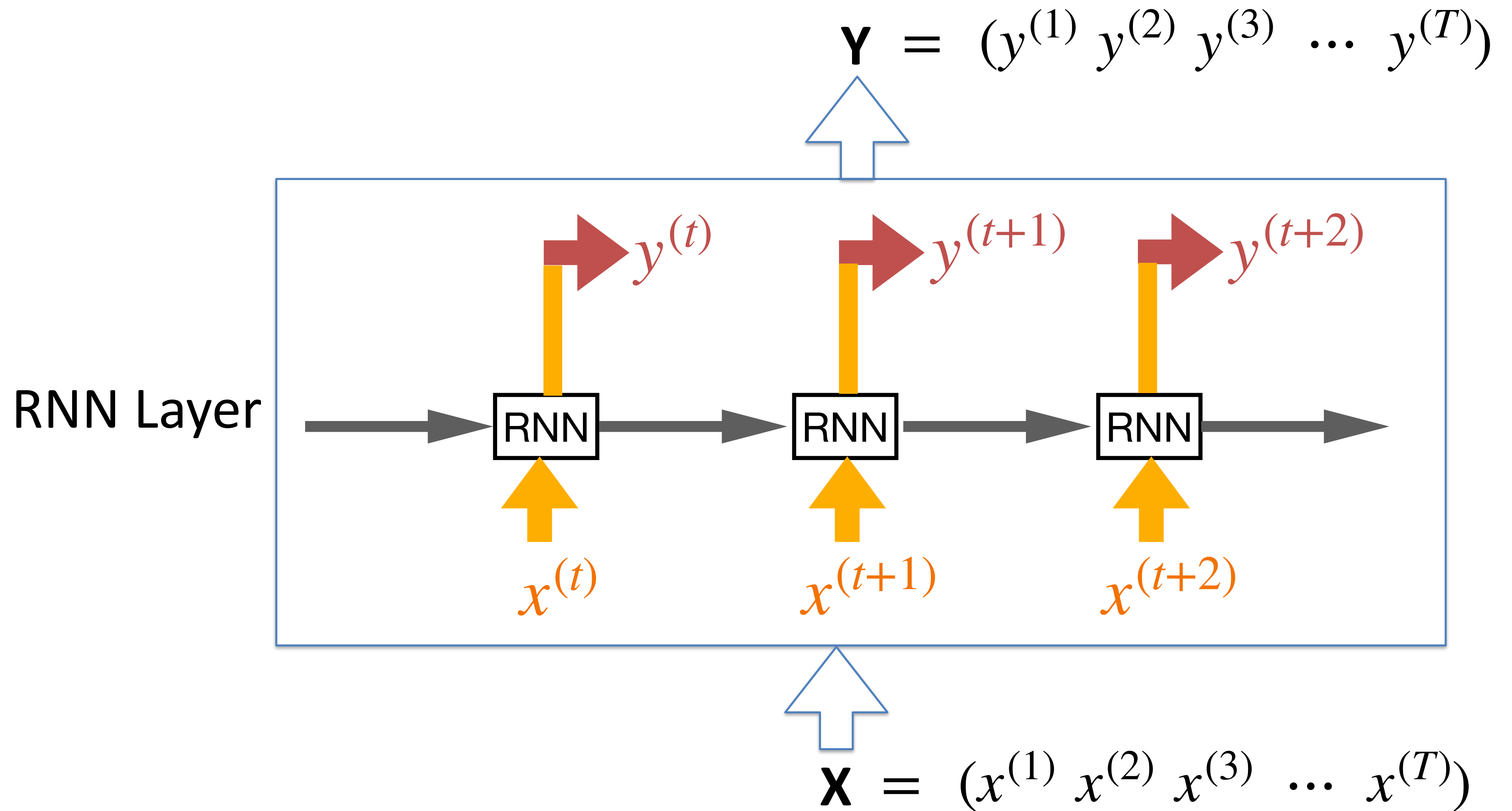
$$\mathbf{Y} = (y^{(1)} \ y^{(2)} \ y^{(3)} \ \dots \ y^{(T)})$$



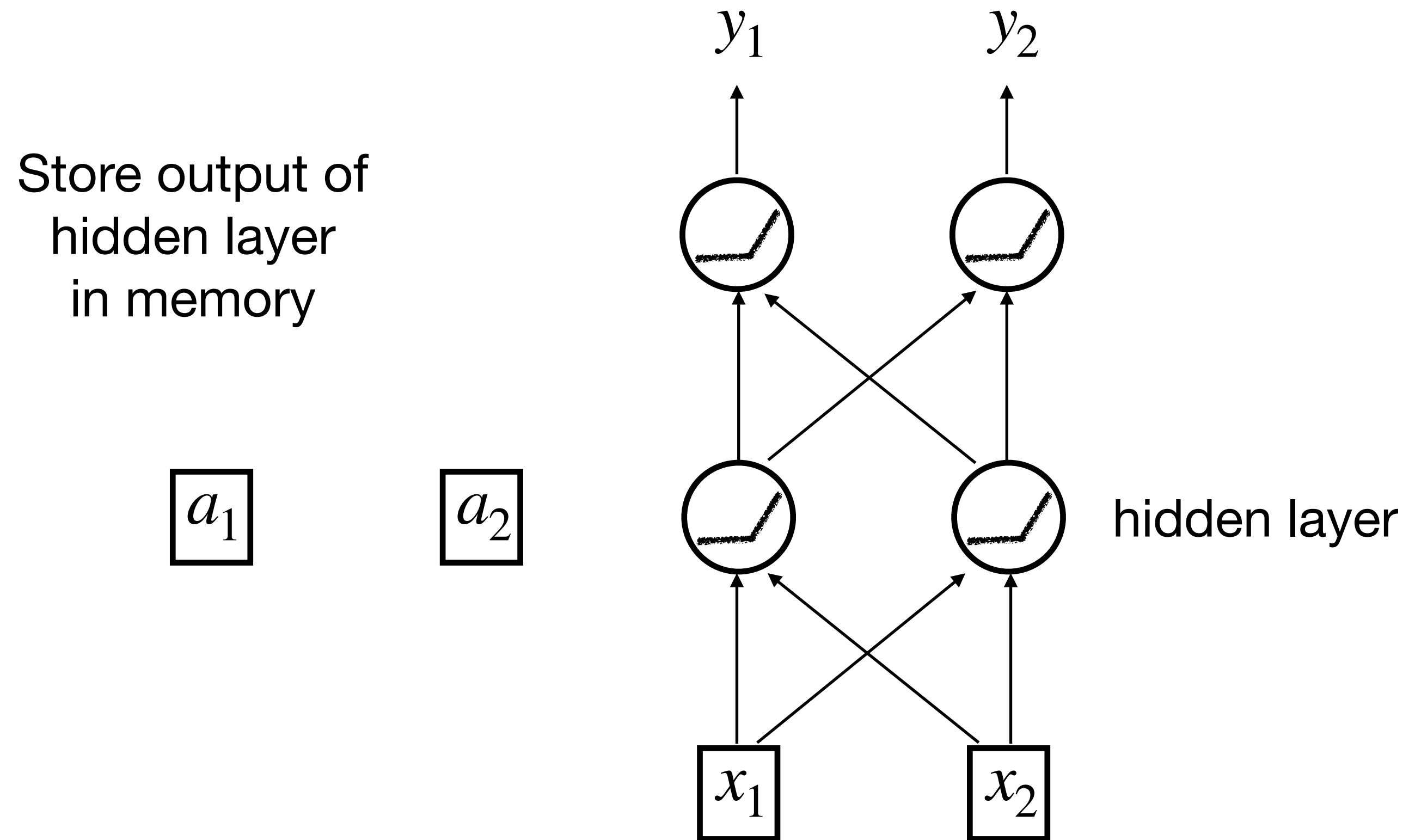
RNN Layer

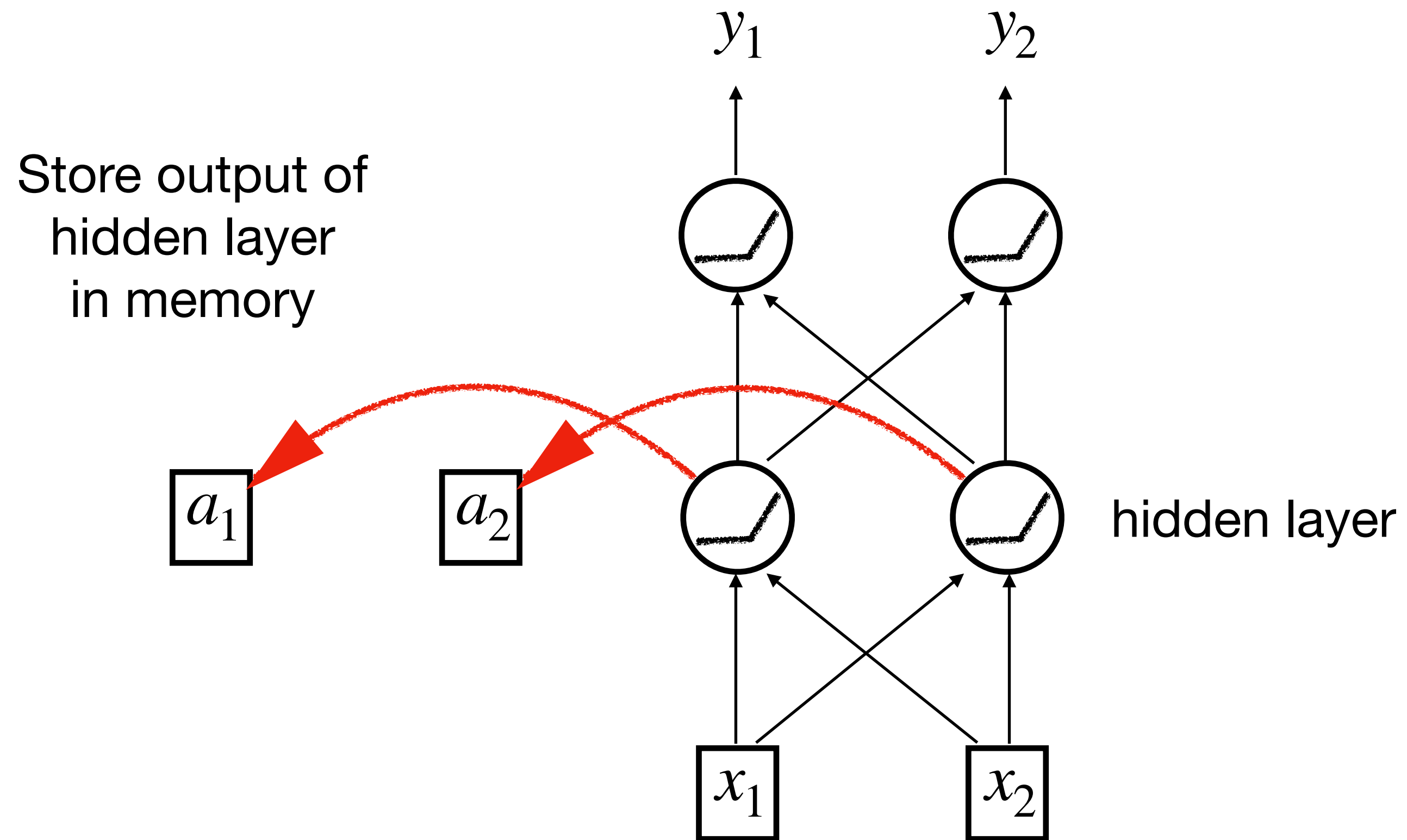


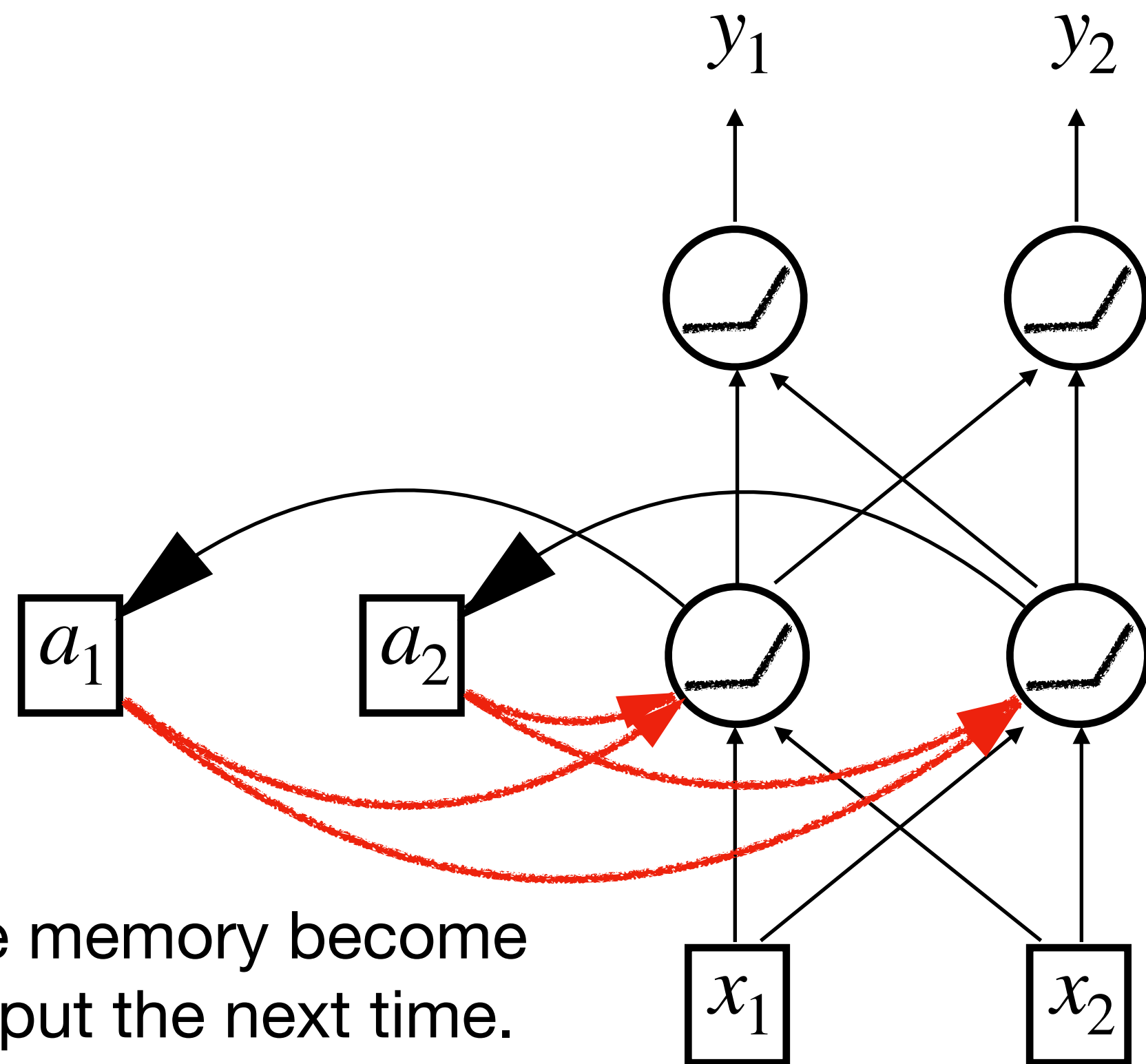
$$\mathbf{X} = (x^{(1)} \ x^{(2)} \ x^{(3)} \ \dots \ x^{(T)})$$



Simple RNN







Data in the memory become
another input the next time.

When data are input as a sequence:

$(x_1^{(1)}, x_2^{(1)})$

$(x_1^{(2)}, x_2^{(2)})$

$\vdots$

$(x_1^{(n)}, x_2^{(n)})$

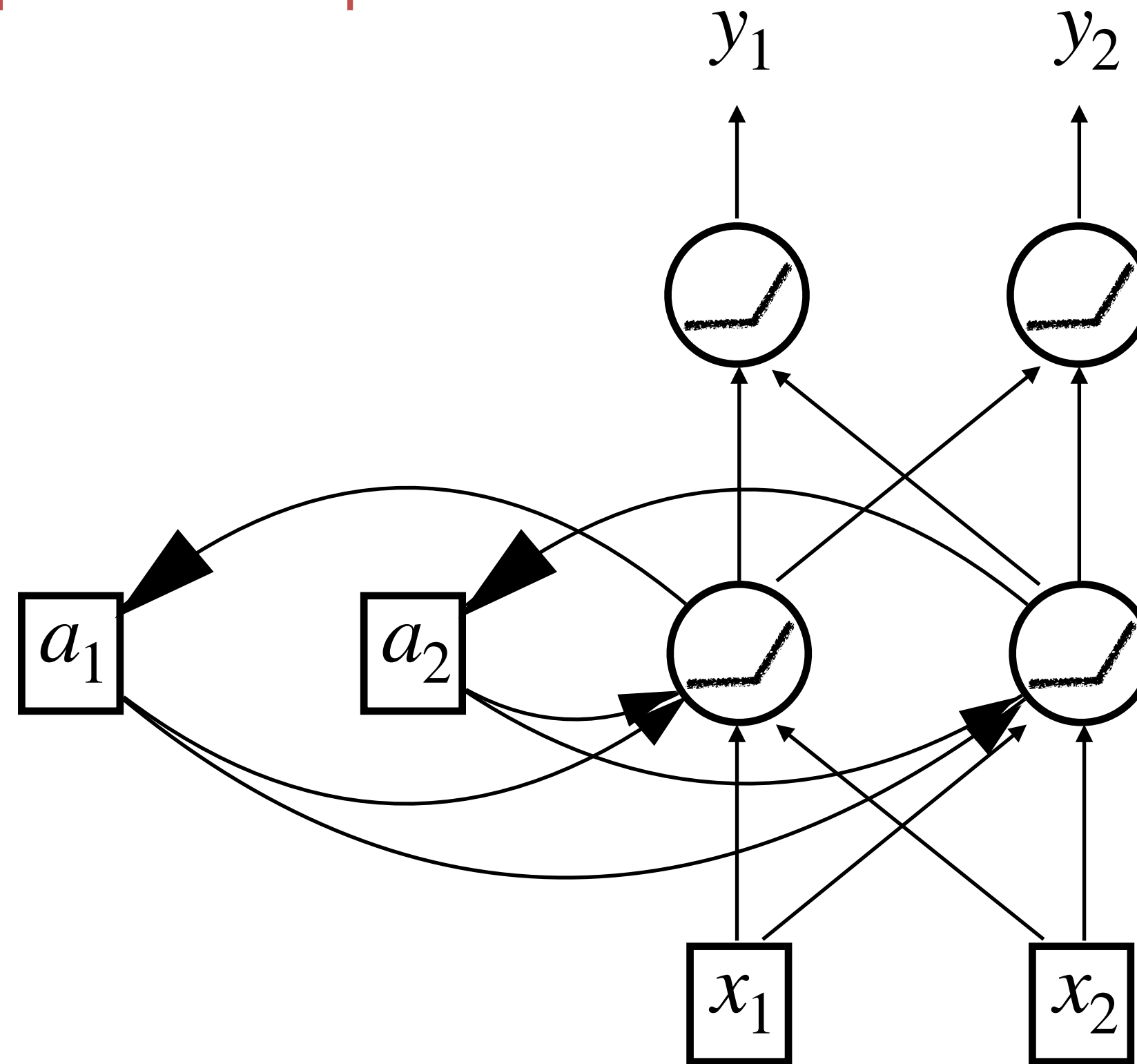
over n steps,
the output is also
a sequence:

$(y_1^{(1)}, y_2^{(1)})$

$(y_1^{(2)}, y_2^{(2)})$

$\vdots$

$(y_1^{(n)}, y_2^{(n)})$



When data are input as a sequence:

$$(x_1^{(1)}, x_2^{(1)})$$

$$(x_1^{(2)}, x_2^{(2)})$$

$\vdots$

$$(x_1^{(n)}, x_2^{(n)})$$

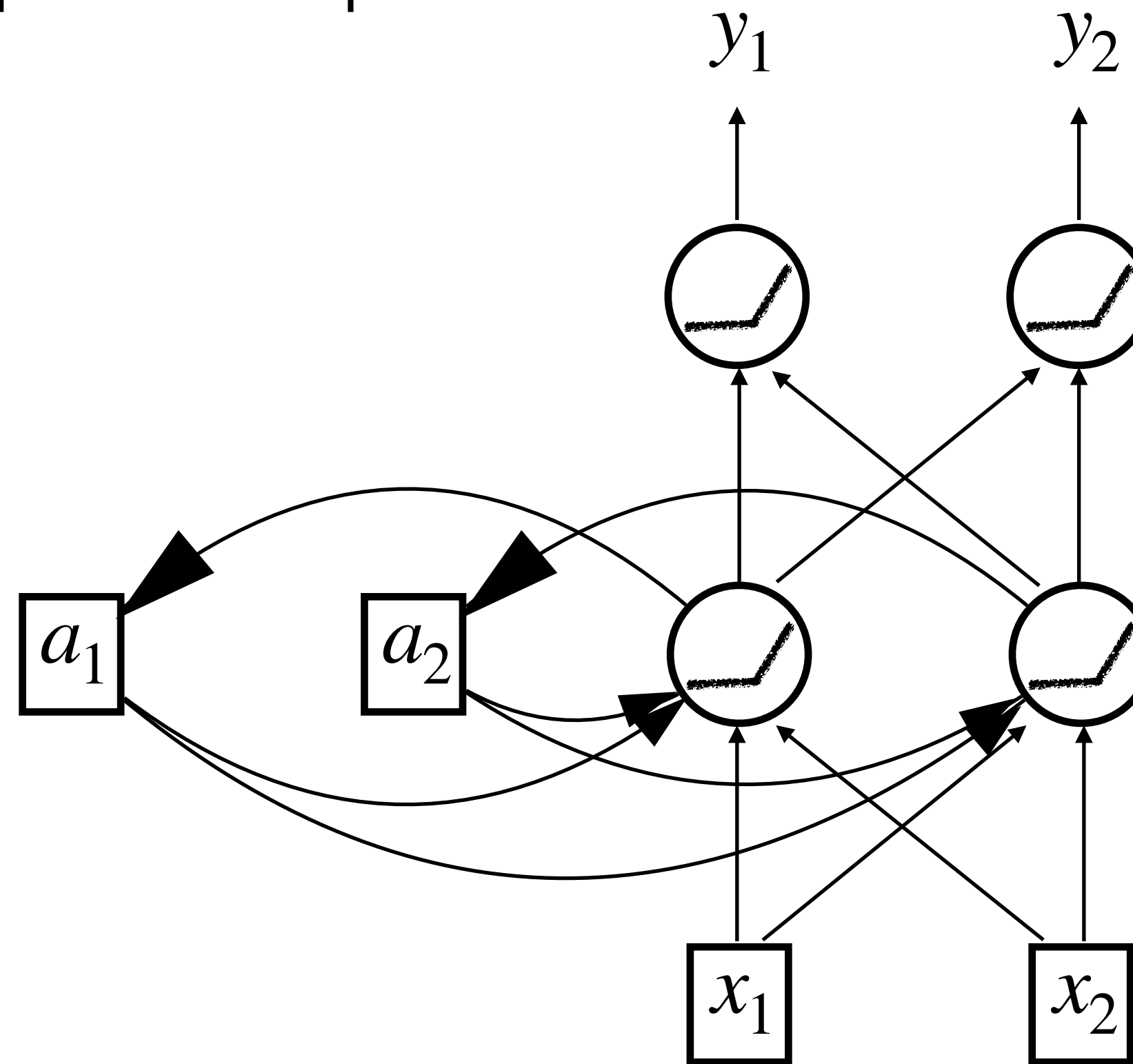
over n steps,
the output is also
a sequence:

$$(y_1^{(1)}, y_2^{(1)})$$

$$(y_1^{(2)}, y_2^{(2)})$$

$\vdots$

$$(y_1^{(n)}, y_2^{(n)})$$



Seen from the outside (of the RNN layer), the input is a matrix, and the output is also a matrix.

When data are input as a sequence:

$(x_1^{(1)}, x_2^{(1)})$

$(x_1^{(2)}, x_2^{(2)})$

$\vdots$

$(x_1^{(n)}, x_2^{(n)})$

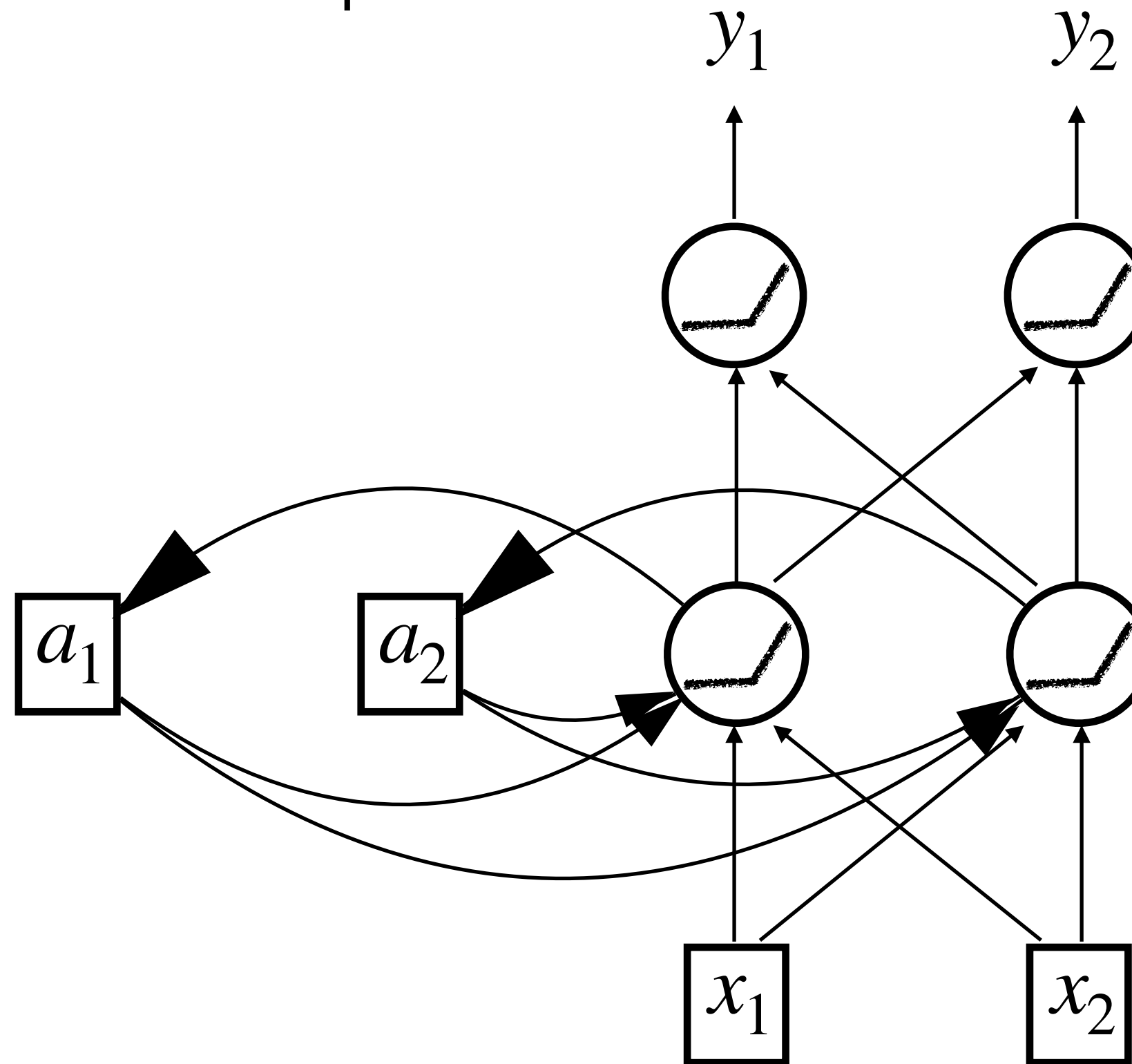
over n steps,
the output is also
a sequence:

$(y_1^{(1)}, y_2^{(1)})$

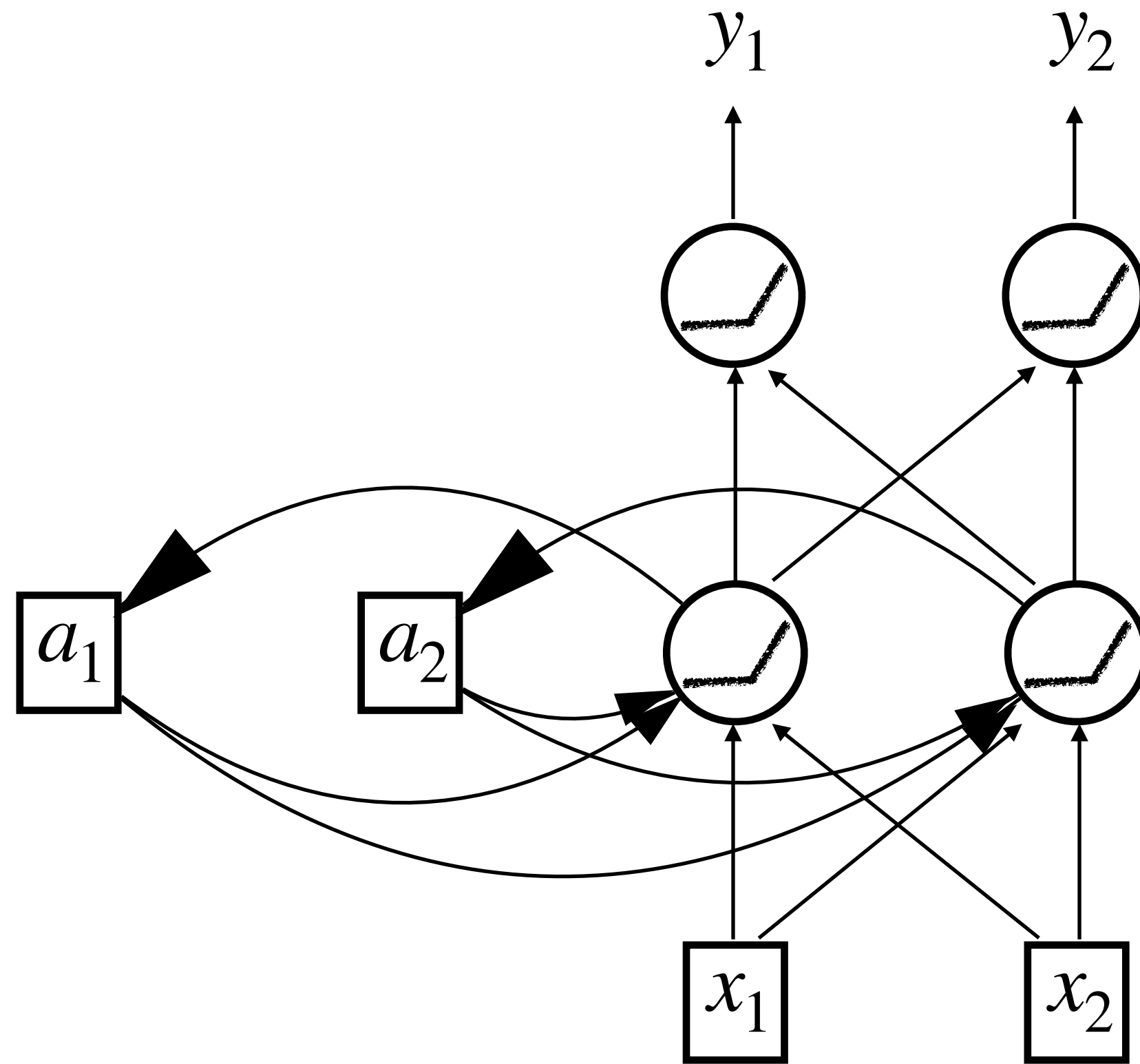
$(y_1^{(2)}, y_2^{(2)})$

$\vdots$

$(y_1^{(n)}, y_2^{(n)})$

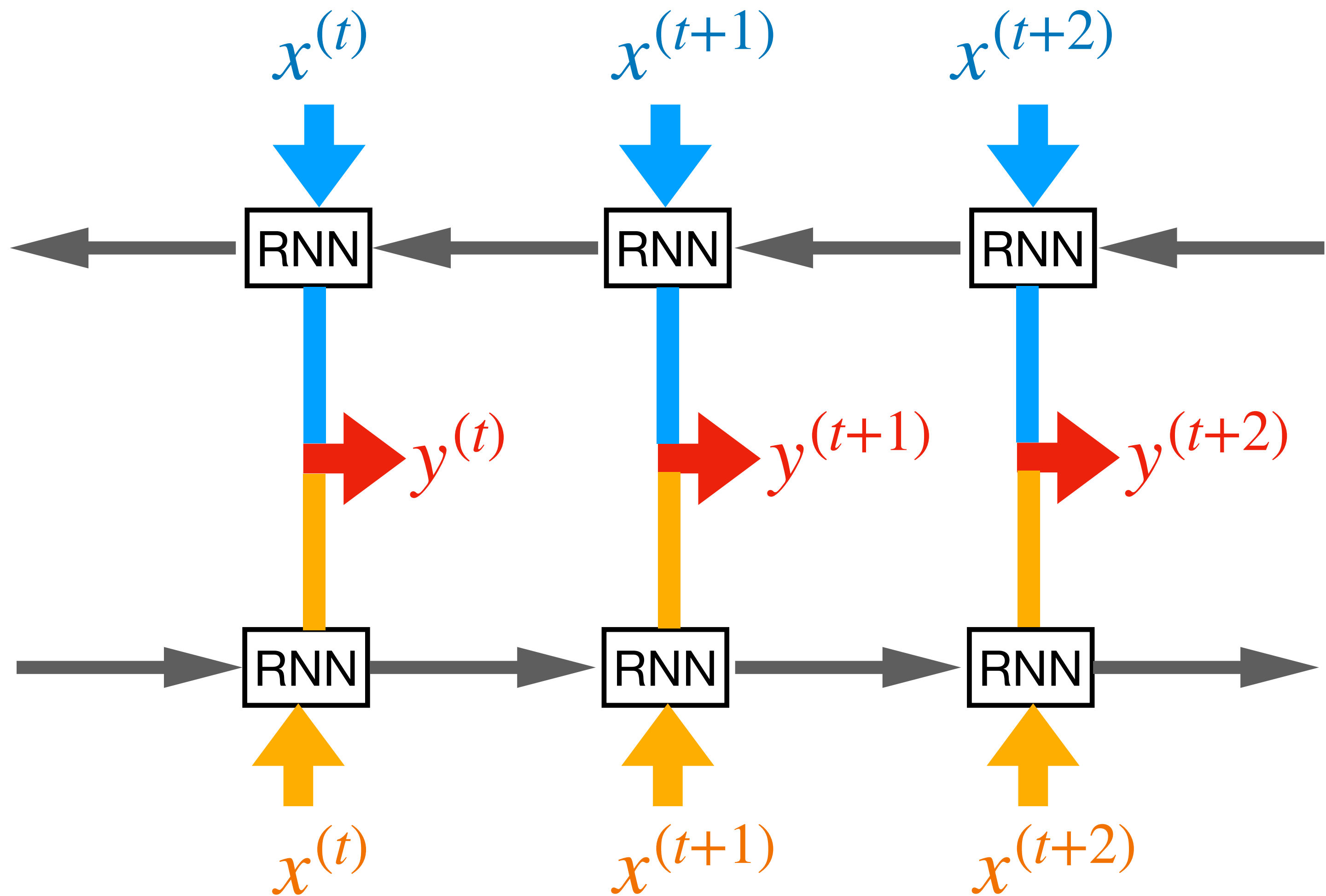


The output at any time step depends not only on the current input, but also the previous inputs in the sequence.

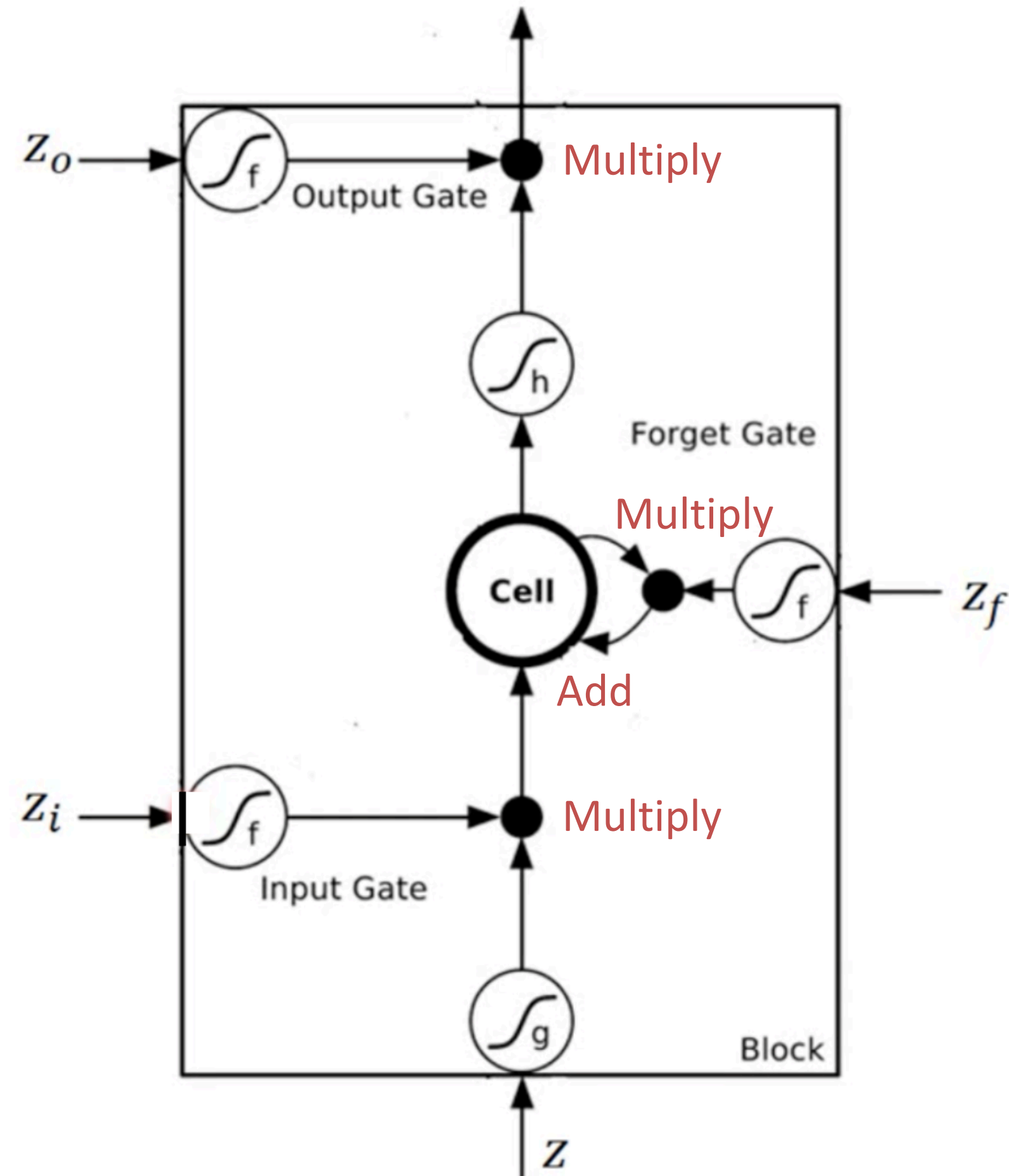


Multiple RNN layers can be stacked.

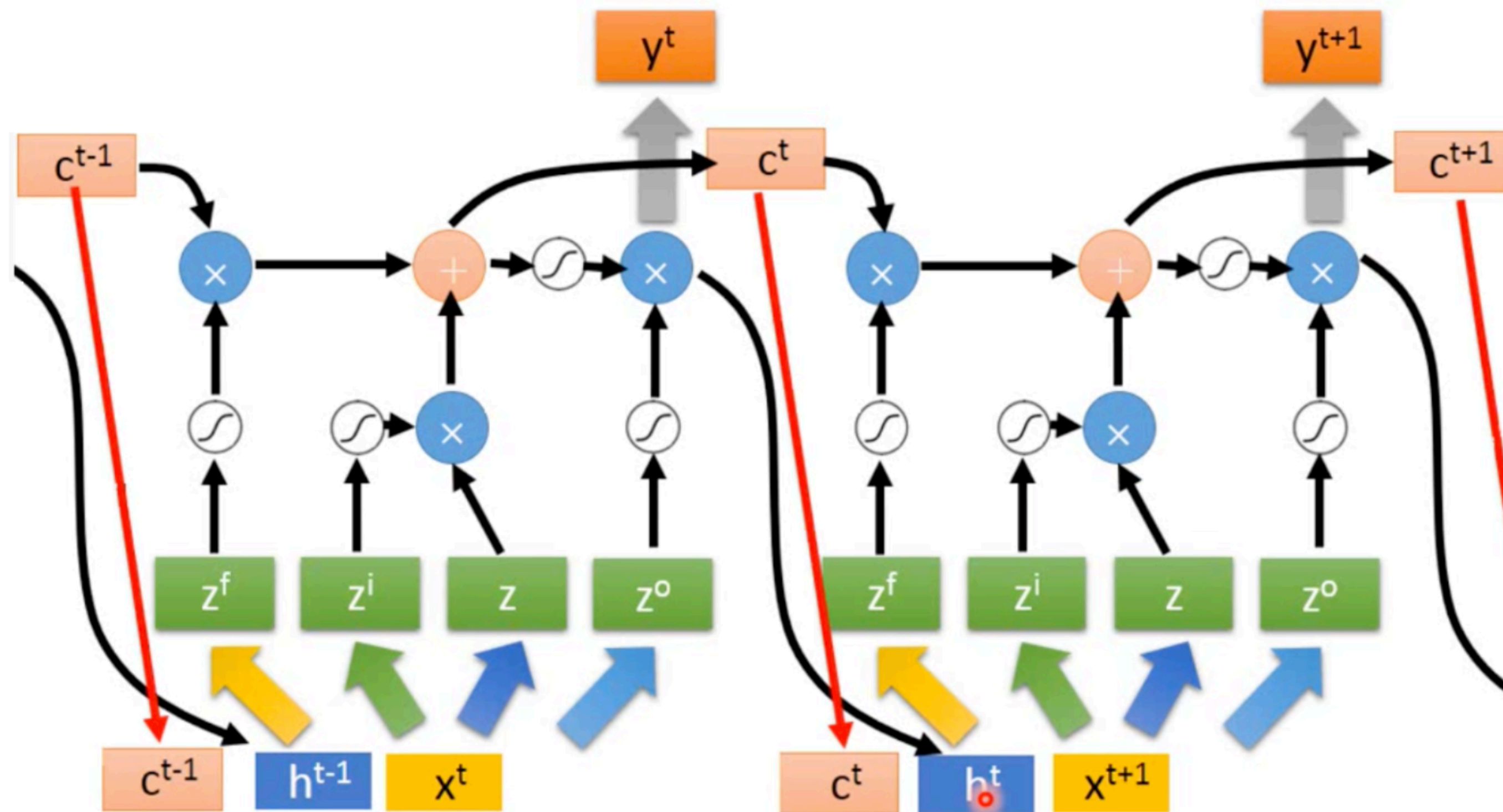
Bidirectional RNN



Long Short-Term Memory (LSTM): a cell



LSTM layer: use LSTM cells to replace normal neurons



Quiz questions:

1. What is a recurrent neural network?
2. What is the use of the “gates” in LSTM?

Roadmap of this lecture:

1. Recurrent neural network (RNN)

2. Use RNN for temperature forecasting

2.1 Try 1: Use an RNN

2.2 Try 2: Use LSTM with recurrent dropout

2.3 Try 3: Stack RNN layers

2.4 Try 4: Use bidirectional RNN



Different types of tasks for **Time-series**

Forecasting: predict what will happen next in a series.

Different types of tasks for **Time-series**

Classification: classify time-series.

Example: given the time-series of the activity of a visitor to a website, classify whether the visitor is a bot or a human.

Different types of tasks for **Time-series**

Event detection: identify the occurrence of a specific expected event within a continuous data stream.

Example: “hot-word detection”, where a model monitors an audio stream and detects utterances like “OK Google” or “Hey Alexa.”

Different types of tasks for **Time-series**

Anomaly detection: detect anything unusual happening within a continuous data stream.

Example: unusual activity on a corporate network, or unusual readings on a manufacturing line.

Task for Regression: **Temperature Forecasting**

Given a time-series of hourly measurements of quantities such as atmospheric pressure and humidity, **predict the temperature in 24 hours.**

Get dataset

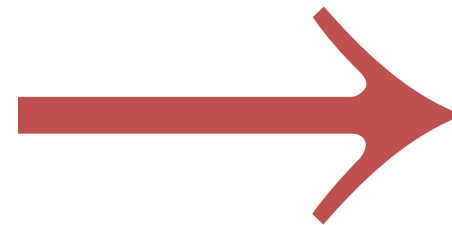
Dataset: recorded at the weather station at Max Planck Institute.

14 quantities (temperature, pressure, humidity, wind direction, and so on) were recorded every 10 minutes over several years.

Use older data for training, newer data for testing



Timeseries



Roadmap of this lecture:

1. Recurrent neural network (RNN)
2. Use RNN for temperature forecasting

2.1 Try 1: Use an RNN

2.2 Try 2: Use LSTM with recurrent dropout

2.3 Try 3: Stack RNN layers

2.4 Try 4: Use bidirectional RNN

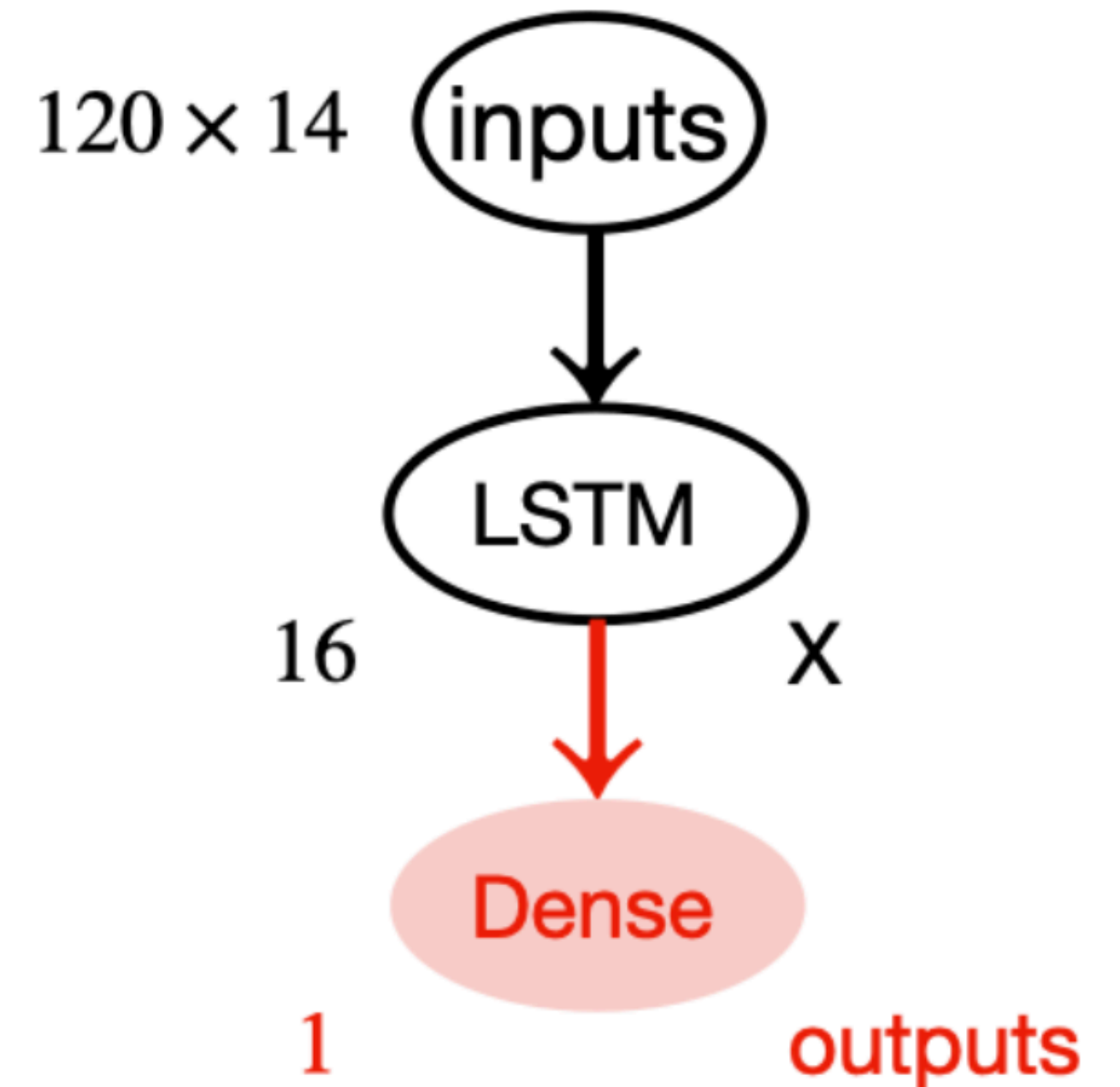


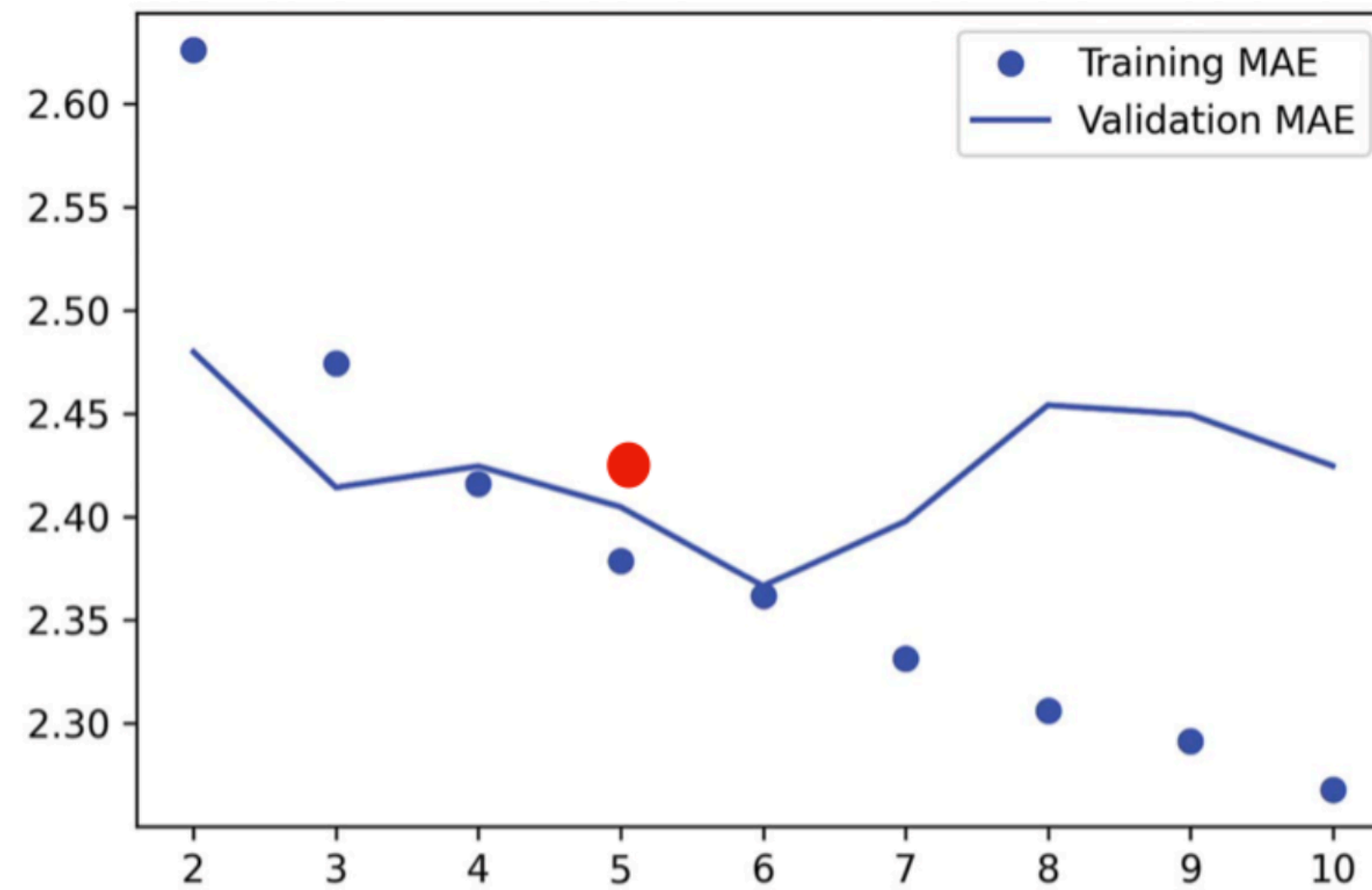
Listing 10.12 A simple LSTM-based model

```
inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1]))
x = layers.LSTM(16)(inputs)
outputs = layers.Dense(1)(x)
model = keras.Model(inputs, outputs)

callbacks = [
    keras.callbacks.ModelCheckpoint("jena_lstm.keras",
                                    save_best_only=True)
]
model.compile(optimizer="rmsprop", loss="mse", metrics=["mae"])
history = model.fit(train_dataset,
                    epochs=10,
                    validation_data=val_dataset,
                    callbacks=callbacks)

model = keras.models.load_model("jena_lstm.keras")
print(f"Test MAE: {model.evaluate(test_dataset)[1]:.2f}")
```





MAE = 2.36 degrees Celsius for validation data

MAE = 2.55 degrees Celsius for test data

Roadmap of this lecture:

1. Recurrent neural network (RNN)

2. Use RNN for temperature forecasting

2.1 Try 1: Use an RNN

2.2 Try 2: Use LSTM with recurrent dropout

2.3 Try 3: Stack RNN layers

2.4 Try 4: Use bidirectional RNN

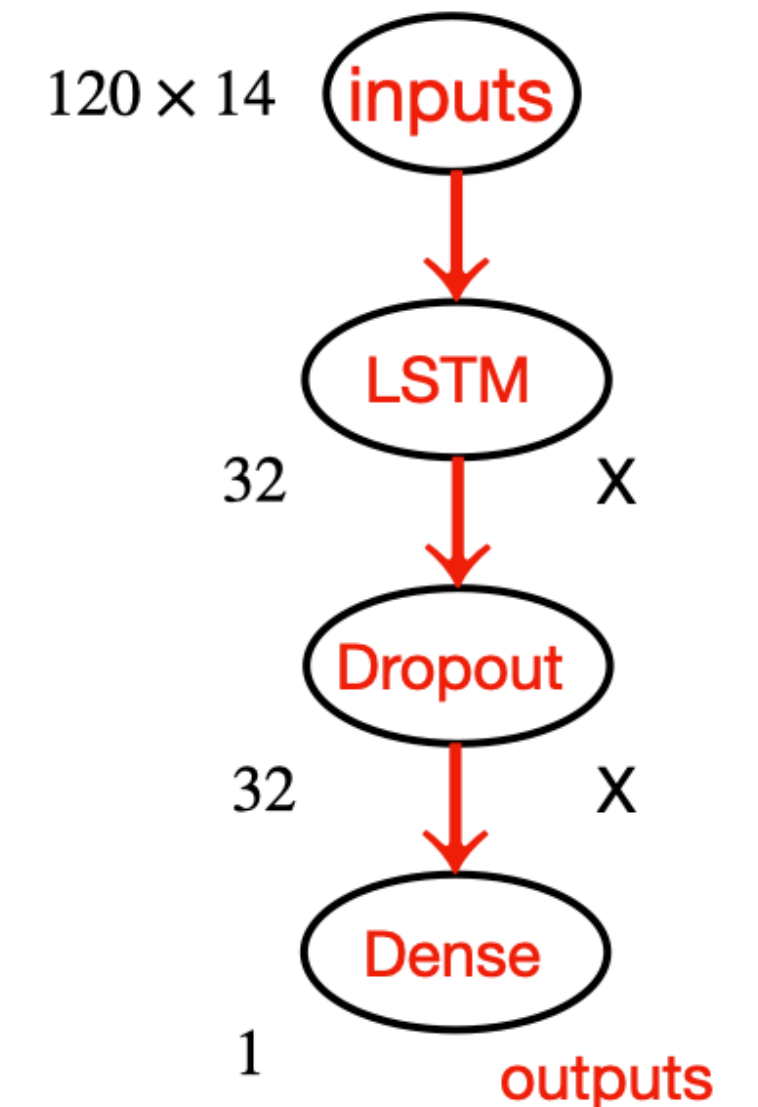


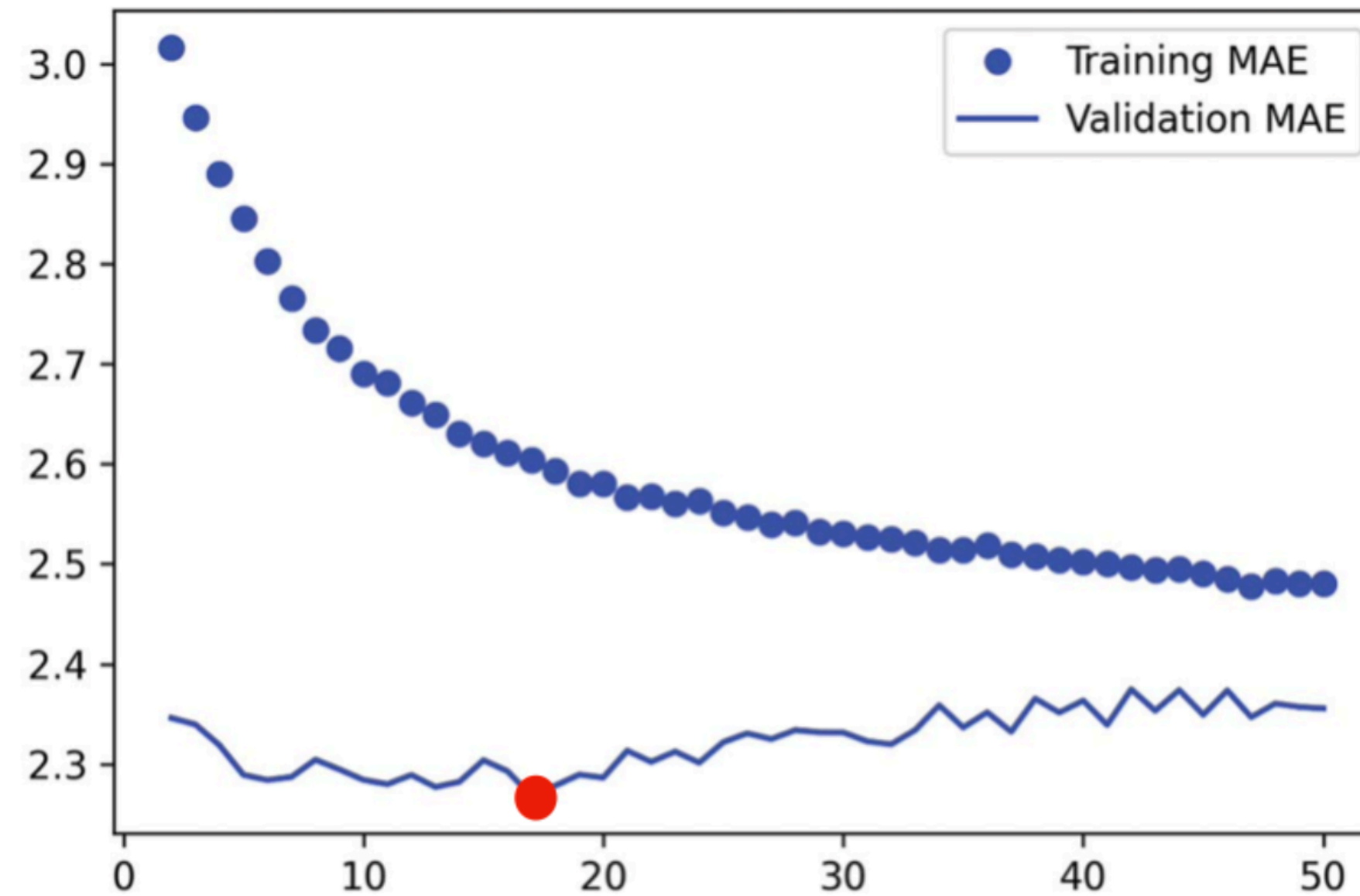
Listing 10.22 Training and evaluating a dropout-regularized LSTM

```
inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1]))
x = layers.LSTM(32, recurrent_dropout=0.25)(inputs)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1)(x)
model = keras.Model(inputs, outputs)

callbacks = [
    keras.callbacks.ModelCheckpoint("jena_lstm_dropout.keras",
                                    save_best_only=True)
]
model.compile(optimizer="rmsprop", loss="mse", metrics=["mae"])
history = model.fit(train_dataset,
                    epochs=50,
                    validation_data=val_dataset,
                    callbacks=callbacks)
```

← To regularize the Dense layer, we also add a Dropout layer after the LSTM.





MAE = 2.27 degrees Celsius for validation data

MAE = 2.45 degrees Celsius for test data

Roadmap of this lecture:

1. Recurrent neural network (RNN)

2. Use RNN for temperature forecasting

2.1 Try 1: Use an RNN

2.2 Try 2: Use LSTM with recurrent dropout

2.3 Try 3: Stack RNN layers

2.4 Try 4: Use bidirectional RNN



Gated Recurrent Unit (GRU)

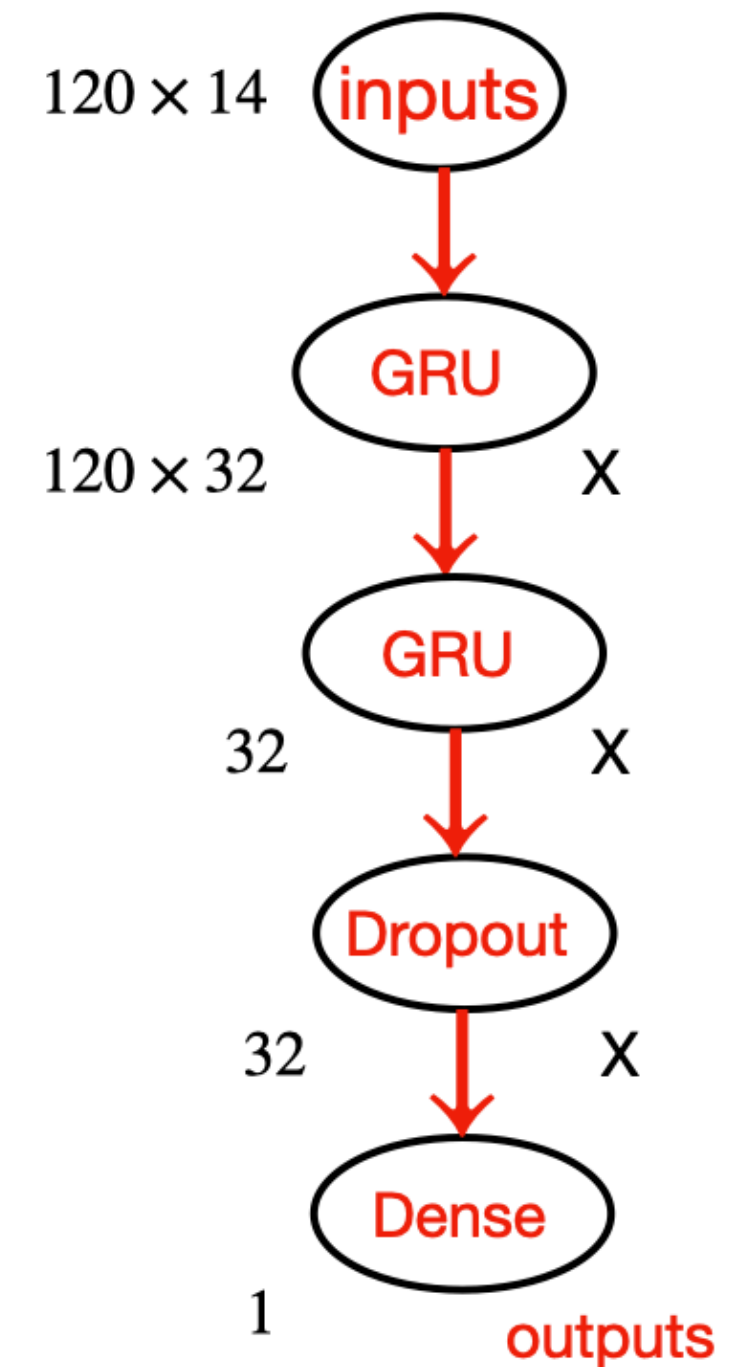
Listing 10.23 Training and evaluating a dropout-regularized, stacked GRU model

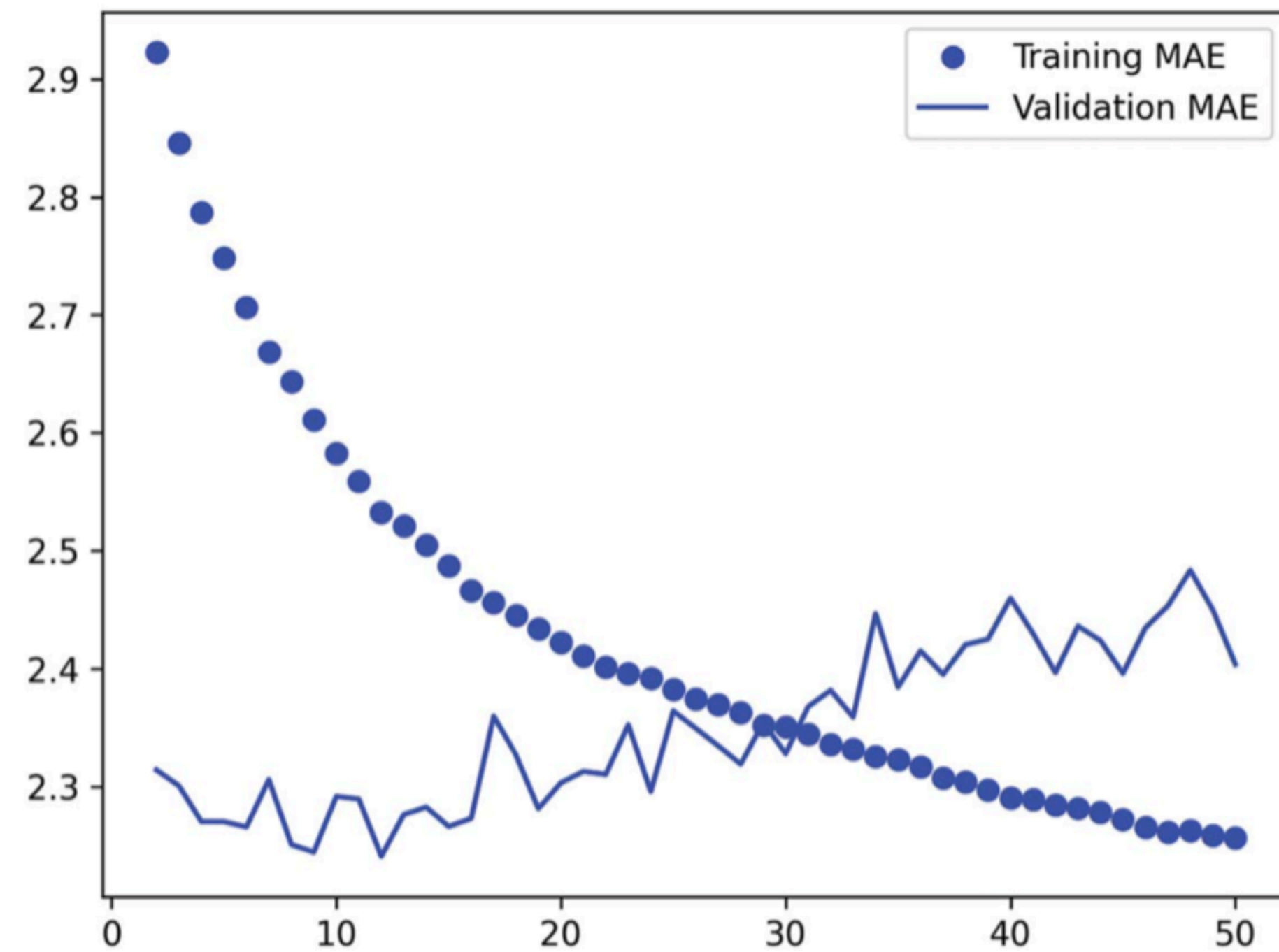
```
inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1]))
x = layers.GRU(32, recurrent_dropout=0.5, return_sequences=True)(inputs)
x = layers.GRU(32, recurrent_dropout=0.5)(x)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1)(x)
model = keras.Model(inputs, outputs)

callbacks = [
    keras.callbacks.ModelCheckpoint("jena_stacked_gru_dropout.keras",
                                    save_best_only=True)
]

model.compile(optimizer="rmsprop", loss="mse", metrics=["mae"])
history = model.fit(train_dataset,
                    epochs=50,
                    validation_data=val_dataset,
                    callbacks=callbacks)

model = keras.models.load_model("jena_stacked_gru_dropout.keras")
print(f"Test MAE: {model.evaluate(test_dataset)[1]:.2f}")
```





MAE = 2.39 degrees Celsius for test data

Roadmap of this lecture:

1. Recurrent neural network (RNN)

2. Use RNN for temperature forecasting

2.1 Try 1: Use an RNN

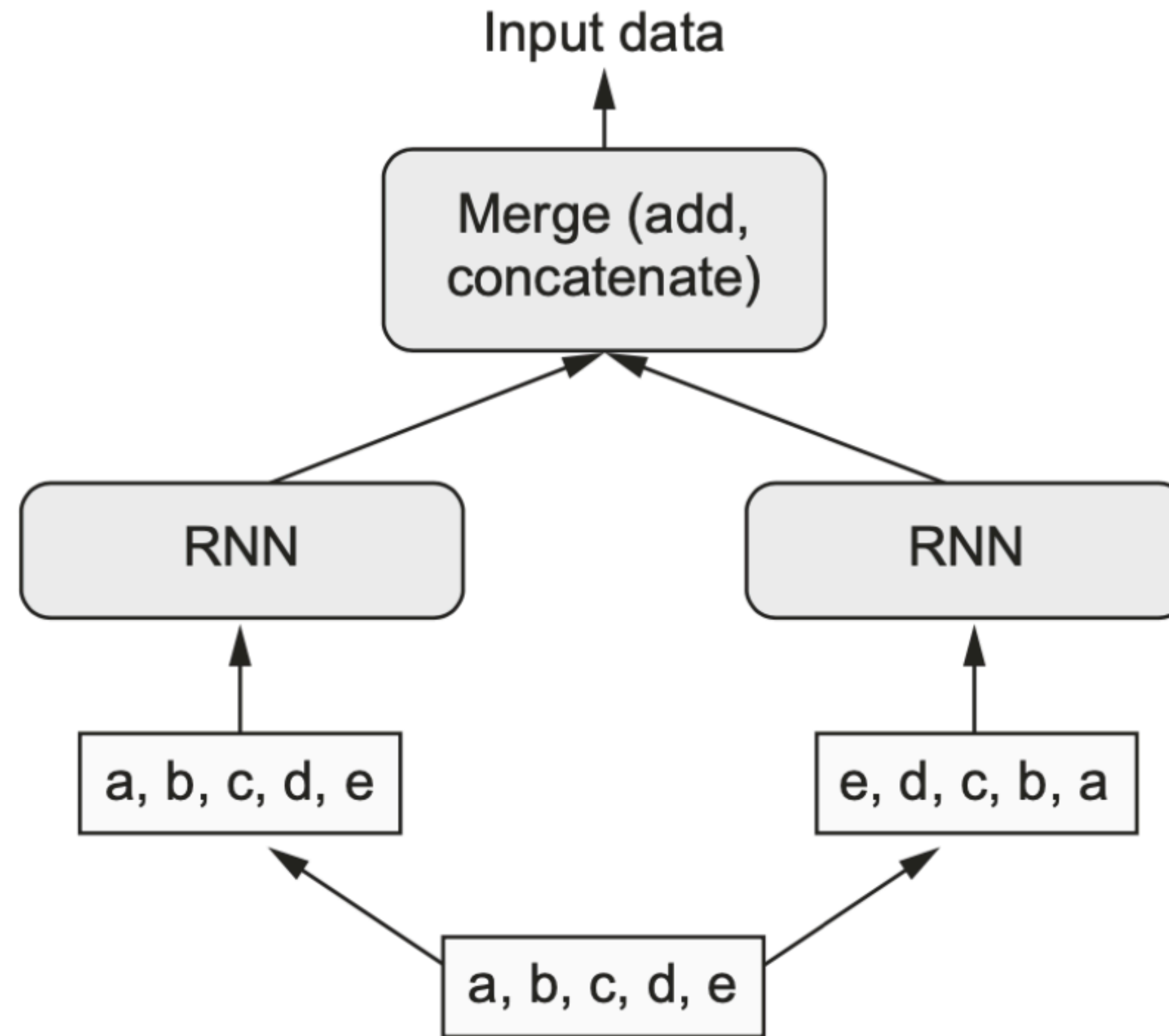
2.2 Try 2: Use LSTM with recurrent dropout

2.3 Try 3: Stack RNN layers

2.4 Try 4: Use bidirectional RNN



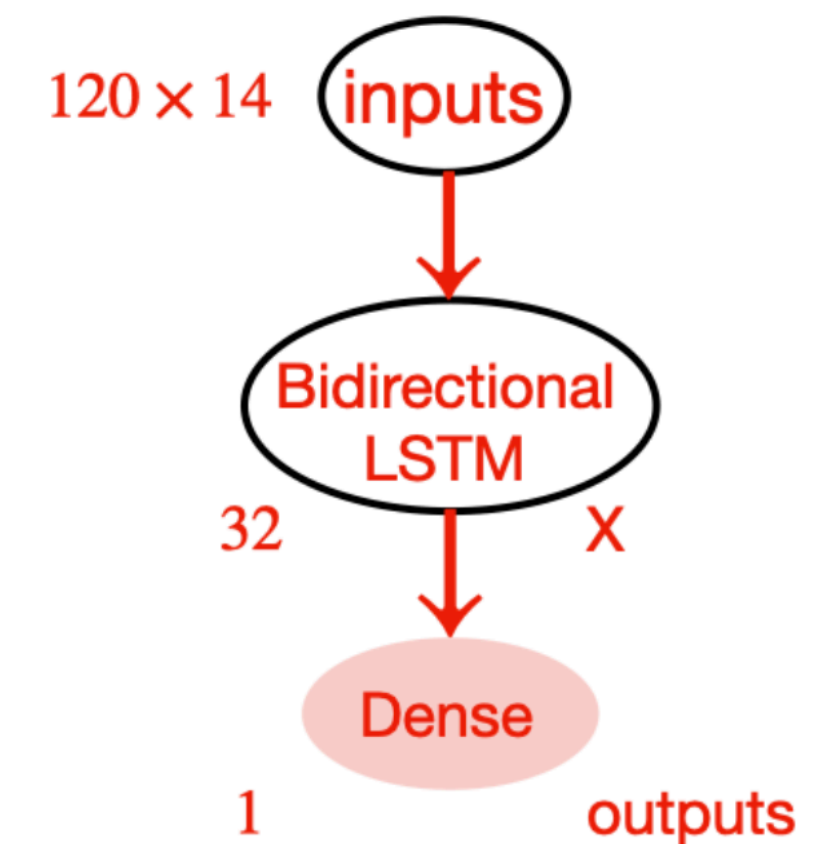
Bidirectional RNN



Listing 10.24 Training and evaluating a bidirectional LSTM

```
inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1]))
x = layers.Bidirectional(layers.LSTM(16))(inputs)
outputs = layers.Dense(1)(x)
model = keras.Model(inputs, outputs)

model.compile(optimizer="rmsprop", loss="mse", metrics=["mae"])
history = model.fit(train_dataset,
                    epochs=10,
                    validation_data=val_dataset)
```



Performance: no better than a simple LSTM.

Quiz questions:

1. Why are RNNs useful for time-series tasks?
2. When is it beneficial to use recurrent dropout, stack RNN layers, or use bidirectional RNN?